

Syncing Variables with Props

Q49

SVELTE 5 REACTIVITY PROPS RUNES

INTERVIEW QUESTION

Question. How do you initialize a reactive local variable that dynamically syncs with an incoming prop using Runes?

ANSWER.

Why `$state` fails here

- **One-time read:** assigning a prop directly to a `**$state**` rune (like `let count = $state(initialCount)`) only captures the initial value. The `$state` rune initializes once during component creation and creates an independent copy.
- **Severed connection:** if the parent component updates the prop later, the child's `$state` variable **will not update**, leaving the UI out of sync with the parent's data.

The correct approach — `$derived`

- **Continuous sync:** to keep a local variable synced with a prop, declare it with the `**$derived**` rune.
- **Automatic recalculation:** anything read synchronously inside the `$derived` expression is tracked as a dependency. When the prop changes, the `$derived` variable is marked as **dirty** (stale) and automatically recalculates, keeping the child component perfectly in sync with the parent.

Optimistic overrides

- **Temporary assignment:** as of Svelte 5.25, `$derived` variables are no

longer strictly read-only. You can reassign them directly to create **optimistic UI** — updating the screen instantly before the server confirms the action.

- **Washed away by truth:** when the server responds and the parent eventually passes down a new prop, the `$derived` expression recalculates. The temporary override is safely replaced by the new **source of truth**.

Post.svelte

```
<script>
  let { post, like } = $props();

  // 1. Syncs automatically with post.likes whenever it
  changes
  let likes = $derived(post.likes);

  async function handleLike() {
    // 2. Optimistic override: instantly update the UI locally
    likes += 1;

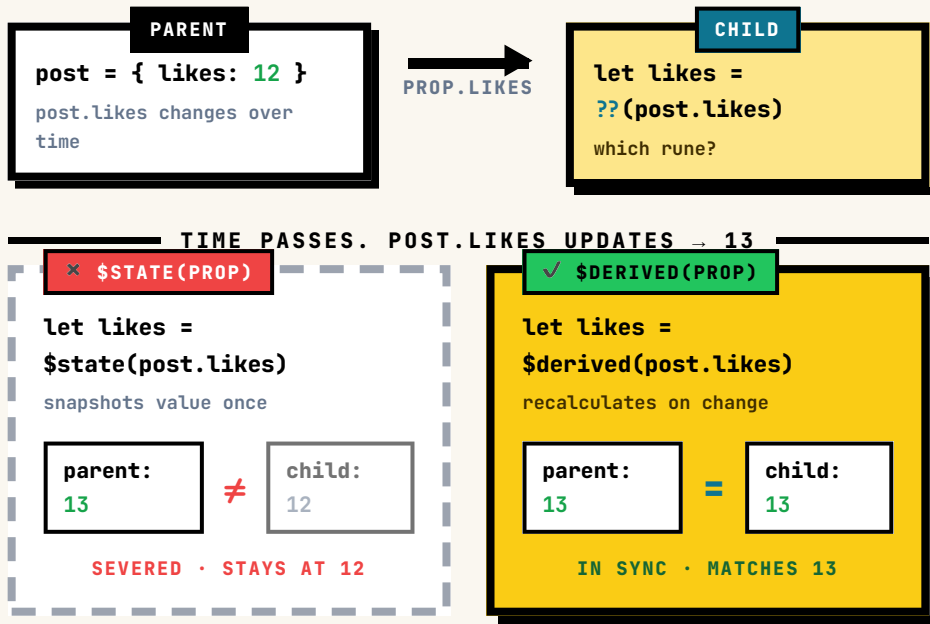
    try {
      await like(); // Tell the server
    } catch {
      likes -= 1; // Rollback if the network request fails
    }
  }
</script>

<button onclick={handleLike}>
  ❤️ {likes}
</button>
```

i Summary. Use `$derived(prop)` rather than

`$state(prop)` to create a reactive local variable that stays synced with an incoming prop. The `$state` rune only reads the prop once during initialization, whereas `$derived` continuously recalculates. You can temporarily reassign the `$derived` variable for optimistic UI updates before the new prop arrives.

TWO WAYS TO HANDLE A PROP



5.25+ BONUS · OPTIMISTIC OVERRIDE

